HashCloak

Code Review and Security Assessment
For
Privacy Cash SDK EVM

Version 2.0: May 26th, 2026

Prepared For:
Privacy Cash

Prepared by:
Manish Kumar | *HashCloak Inc.*

Table Of Contents

Executive Summary

Privacy Cash engaged *HashCloak Inc*. to perform a code review and security assessment for Privacy Cash SDK EVM, an SDK for interacting with the EVM version of Privacy Cash inspired from Tornado Cash, supporting private ETH transfers on Base.

During the audit period, we familiarized ourselves with the overall concepts of Tornado Cash and its core components relevant to the audit scope by reviewing the code and the provided documentation. We assessed security vulnerabilities such as input validation, authentication, crypto/secrets handling, data exposure, use of strong randomness. Subsequently, we examined the code for errors, primarily focusing on logical correctness and identifying potential issues that could lead to loss of funds, or other privacy-related issues. The code was found to be adapted from Tornado Cash.

Later, PR #20 was raised which adds ETH and USDT pools support on Ethereum mainnet alongside the existing Base network. The core change is defining the `NetworkConfig` interface that replaces hardcoded Base specific constants with a per-network configuration object. The PR also handles USDT's non-standard ERC20 approval behaviour that requires resetting an existing allowance to zero before setting a new one. The gas estimation logic is also replaced from static limits to dynamic `estimateGas` calls. We did not find any issue in the submitted PR and all the intended changes were found to be in place. The changes were merged at commit 8fbc4ba2889f915f0f00fefa097a6e4acd4a6a60.

Overall, we found the source code is of good quality in representing the entities in the specification. The code is modular and well-structured.

We only found Low issues:

Severity	Number of Findings	Status
Critical	0	N/A
High	0	N/A
Medium	0	N/A
Low	2	PSM-1 and PSM-2 acknowledged
Informational	0	N/A

Version Info

Versions	Delivery Date
Final Delivery	April 29th, 2026
Version 2.0	May 26th, 2026

Scope

For the audit, we considered the following repositories:

- https://github.com/Privacy-Cash/privacy-cash-sdk-evm at commit c3d7ae184df05d2ffbc5f85ecb082937a0512ef0

With the following files in scope:

- privacy-cash-sdk-evm
  - src/*

Overview

Privacy Cash Core EVM is an EVM version of Privacy Cash inspired from Tornado Cash, supporting private ETH transfers on Base. The protocol implements a privacy-preserving transaction system, drawing inspiration from Tornado Cash but adapted for the Base environment. It allows users to privately deposit and withdraw funds by leveraging zero-knowledge proofs (specifically Groth16) and a Merkle tree structure. Each deposit generates a unique commitment that is inserted into a Merkle tree maintained on-chain. Withdrawals are performed by proving knowledge of a valid note inside the tree using a zero-knowledge proof and corresponding nullifier to prevent double-spending. The SDK allows you to interact with the Core EVM.

Methodology

The audit was conducted through a combination of manual verification. The primary focus was to map the available specifications and existing resources such as Tornado Nova to the actual implementation. We also assessed the code for sufficient code coverage and various types of vulnerabilities, including:

- Input validation
- Secret handling
- Merkle root validation
- Use of strong randomness
- Deposit and withdrawal logic
- Look for any dead code
- Any attack that can impact user funds
- Any undefined behavior
- Any transaction replay attack or double spending possibility

Overview of Evaluated Components

privacy-cash-sdk-evm

- Use of cryptographic randomness
- Encryption/decryption of UTXOs
- Input validation
- Data exposure and logging
- DoS
- Race conditions
- Authentication tag handling
- Key derivation from signature
- Secret handling
- Zk-circuit input integrity
- Use of centralized indexer

Findings

PSM-1: Generation of Encryption keys and UTXOs Keypair from signature

Type: Low
Files affected:

- src/utils/encryption.ts

Description: The function `deriveKeys` is used to derive the encryption keys and the UTXO's keypair from signature which is generated by wallet signing a constant message: `SIGN_PRIVACY_MESSAGE = 'Privacy Money account sign in';` and then hashing the signature. While this approach may function correctly in the current context, it introduces avoidable risk. If any third party site tricks a user into signing the same message with their wallet, that exposes its encryption keys and thus drains all the privacy pool funds for that user. Although phishing attacks cannot be fully mitigated, the current design makes such attacks easier to execute against PrivacyCash users.

Impact: Signing the same message anywhere else can result in exposing of the encryption key. This can lead to funds associated with that key to be drained from the associated privacy cash pool.

Recommendation: Ideally, the keys should be generated in a cryptographically random fashion with no dependency between the encryption keys and UTXO's keypair. Since we need a deterministic way, a cryptographic Key-Derivation Function should be used to generate new keys using the wallet private keys.

Below is a snippet on how encryption keys can be safely generated using a key derivation function:

```
import crypto from 'crypto';

crypto.hkdf(
  'sha256',         // digest
  Buffer.from(key),   // input key material (IKM)
  Buffer.from(salt),
  Buffer.from(info),
  32,               // output length
  (err, derivedKey) => {
    if (err) throw err;
    return derivedKey;  // derived key
  }
);
```

Where `key` is the wallet private key given as input key material (ikm), `salt` can be considered as a domain separator which can be given something as `salt='privacyCashEncryptionKey'` and `info` can be given something as `info='privacyCashV1'`. This generates a cryptographically strong secret key of 32 byte which can be used for encryption. Using different salts, different keys can be generated for encryption of the UTXOs.

Status: The team acknowledged it but decided to go with the current approach. The user must ensure they don't sign the same message on any phishing site which can result in exposing their encryption keys.

PSM-2: Trust and dependence on Centralized indexer

Type: Low
Files affected:

- src/utils/utils.ts
- src/deposit.ts
- src/withdraw.ts

Description: The SDK routes several operational responsibilities to a single off-chain centralized indexer service at `INDEXER_URL` without any sanity checks or on-chain verification and treats its responses as fully trusted. The Merkle root and `nextIndex` from GET /merkle/root, the `index` and `pathElements` from POST /commitment/, the encrypted UTXO set from GET /get_encrypted), the deposit/withdraw confirmation from POST /check_encrypted_output and the prices, minimum deposit and withdrawal, rent fees, and fee_rate returned by GET /config are all consumed without any cross-check against on-chain state. This also creates a linkability of spend to caller as the indexer learns which commitment a particular caller intended to spend which results in reduced privacy and unlinkability.

Impact: Results in UTXO de-anonymization, weakening privacy and unlinkability guarantees. By observing which encrypted UTXO ranges a client queries immediately before constructing a withdrawal proof, the indexer can correlate user activity and potentially link deposit and withdrawal addresses.

Recommendation: Verify the merkle root on-chain is a known root and provide a fallback that rebuilds the merkle tree locally to derive the root and path elements.

Status: The team acknowledged the issue. Since all data is emitted on-chain, anyone wanting to run an indexer/relayer can spin up their own.

References

- Tornado Nova
- Privacy Cash